

PROGRAMMING ASSIGNMENT 1: Q-LEARNING FOR ACROBOT

1. Introduction

This report documents a tabular Q-learning solution for the Acrobot-v1 reinforcement learning environment. Acrobot is a two-link pendulum control task where the agent must swing the free end high enough to reach the target height. The environment provides a continuous six-dimensional observation and a discrete action space with three torque actions.

Q-learning was selected because it is a standard value-based reinforcement learning algorithm for discrete action problems. Since Acrobot observations are continuous, the implementation discretizes the state space before storing and updating values in a Q-table.

2. Environment Description

- Environment: Acrobot-v1
- Observation space: six continuous values
- Action space: three discrete torque actions
- Reward structure: the agent receives -1 per step until the episode terminates
- Objective: reach the goal in as few steps as possible

The observation vector contains trigonometric terms for the link angles and angular velocity terms. The action space is already discrete, so only the observation space requires conversion before tabular Q-learning can be used.

3. Methodology

The agent uses tabular Q-learning with an epsilon-greedy exploration policy. During training, the agent sometimes chooses random actions to explore and otherwise chooses the action with the highest Q-value for the current discretized state.

The Q-learning update rule is:

$$Q(s, a) \leftarrow Q(s, a) + \alpha [r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

where α is the learning rate, γ is the discount factor, r is the reward, s is the current state, a is the selected action, and s' is the next state.

4. State Discretization

Acrobot-v1 has a continuous observation space, so the implementation converts each observation into a tuple of bin indices. The four sine/cosine observation dimensions are clipped to $[-1, 1]$. The two angular velocity dimensions are clipped to practical Acrobot velocity bounds.

The discretization uses the following number of bins:

Observation dimensions Bins

$\cos(\theta_1)$, $\sin(\theta_1)$, $\cos(\theta_2)$, $\sin(\theta_2)$ 8 each

angular velocity 1, angular velocity 2 12 each

This creates a Q-table with shape (8, 8, 8, 8, 12, 12, 3), where the final dimension stores one Q-value for each action.

5. Hyperparameters

Hyperparameter Value

Training episodes 5000

Maximum steps per episode 500

Learning rate α 0.1

Discount factor γ 0.99

Initial epsilon 1.0

Minimum epsilon 0.05

Epsilon decay 0.995

Evaluation episodes 100

Evaluation interval 500 episodes

Moving average window 100 episodes

Random seed 42

6. Performance Metrics

The following metrics were selected so that Q-learning can be compared fairly with future reinforcement learning algorithms for the same Acrobot problem:

- Episode return: total reward collected in one episode.
- Moving-average return: smoothed reward trend over a 100-episode window.
- Episode length: number of steps before termination or truncation.
- Training success flag: whether the episode reached the goal.
- Checkpoint success rate: greedy-policy success rate measured every 500 episodes.
- Final evaluation mean reward and standard deviation.
- Final evaluation success rate.
- Final evaluation mean episode length.
- Training time.

- Sample efficiency threshold crossing.

7. Results

The trained Q-learning agent was evaluated using a greedy policy over 100 evaluation episodes.

Metric Result

Total training episodes 5000

Final epsilon 0.05

Best 100-episode moving average reward -242.72

Final evaluation mean reward -261.30

Final evaluation reward standard deviation 92.24

Final evaluation success rate 96.0%

Final evaluation mean episode length 262.3 steps

Training time 407.2 seconds

Sample efficiency threshold episode Not reached

The checkpoint evaluation success rate increased substantially during training. It began at 59.0% after 500 episodes and reached 96.0% after 5000 episodes, with a peak checkpoint success rate of 100.0% at 3500 episodes.

8. Visualizations

The script generates four plots to demonstrate training and evaluation performance:

- Reward curve: raw episode return and 100-episode moving average.
- Episode length curve: raw episode length and 100-episode moving average.
- Checkpoint success-rate curve: greedy-policy success rate at evaluation checkpoints.
- Epsilon decay curve: decrease in exploration rate during training.

These plots are included in the following pages of the PDF submission.

9. Discussion

The results show that the discretized Q-learning agent learned a useful policy for Acrobot. Since the reward is -1 per step, better policies should produce less negative returns and shorter episodes. The final greedy evaluation success rate of 96.0% indicates that the learned Q-table usually reaches the goal.

The main limitation is that tabular Q-learning depends heavily on the discretization scheme. A coarse discretization can hide important state differences, while a very fine discretization creates a much larger Q-table and may require more training. Future improvements could test different bin counts, slower epsilon decay, additional training episodes, or function approximation methods.

10. Conclusion

The Python implementation successfully trains a Q-learning agent for Acrobot-v1, records reusable performance metrics, saves CSV outputs, and generates visualizations that demonstrate learning progress. The final evaluation shows strong performance with a 96.0% success rate over 100 greedy evaluation episodes.

Figure 1: Reward Curve

Generated by q_learning_acrobot.py during the completed training run.

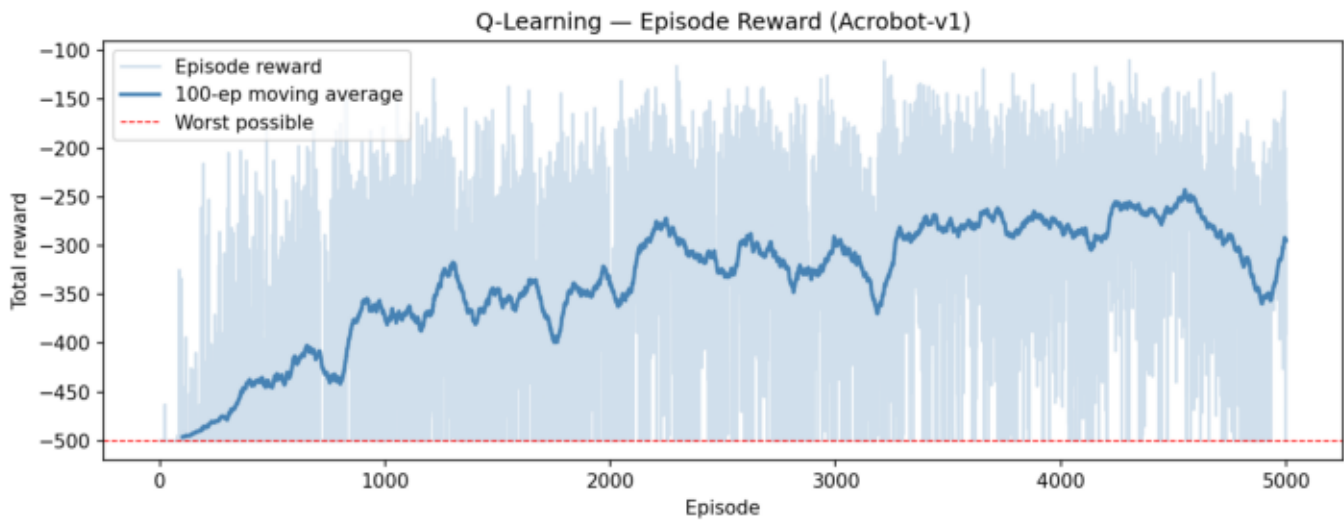


Figure 2: Episode Length Curve

Generated by q_learning_acrobot.py during the completed training run.

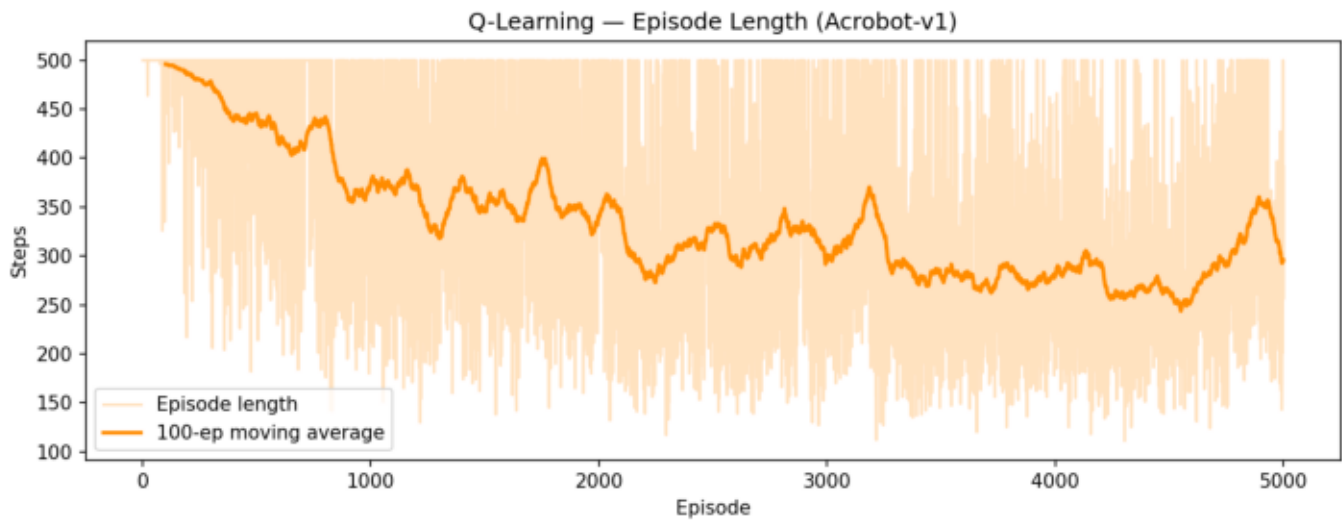


Figure 3: Success Rate

Generated by q_learning_acrobot.py during the completed training run.

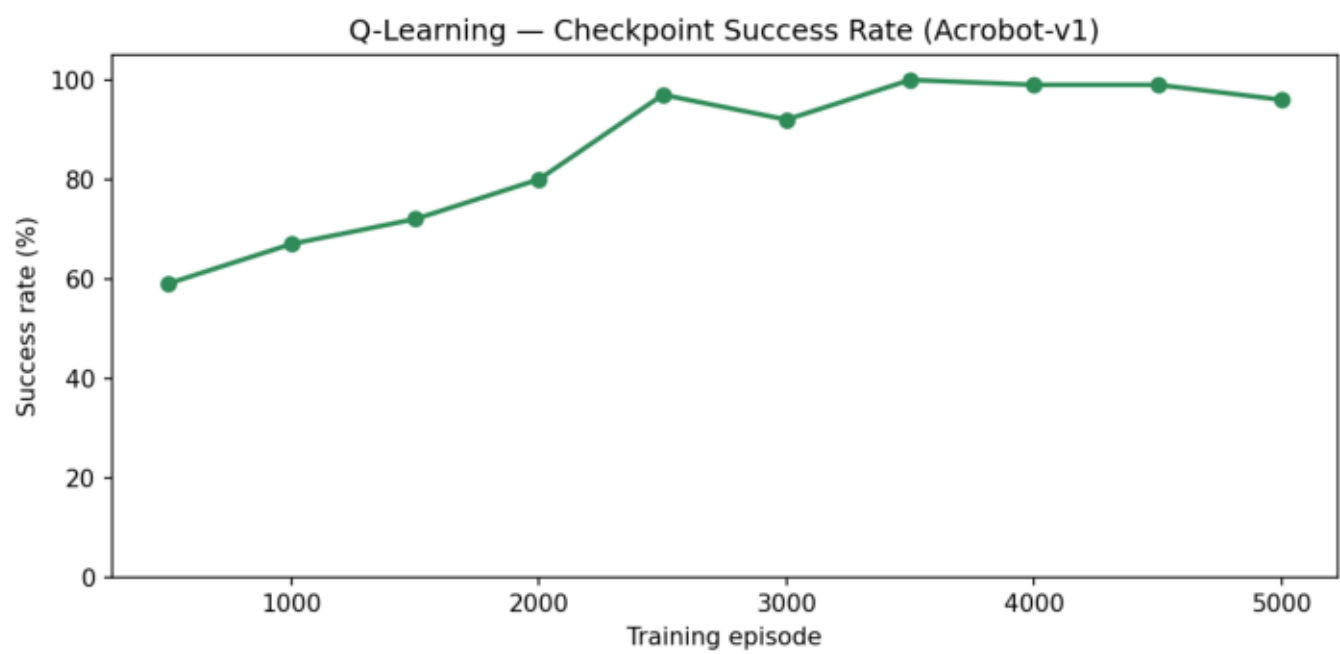


Figure 4: Epsilon Decay

Generated by q_learning_acrobot.py during the completed training run.

